UNIVERSIDAD SANTO TOMÁS

Sistema colaborativo de robots móviles para asistencia de orientación en entornos interiores con múltiples pisos

Realizado por:
Santiago Hernández Ávila
Jonny Alejandro Mejía León

Informe de Avance - Semana 12
Fase 3: Cierre de diseño conceptual y validación experimental de plataforma

Dirigido por:
Ing. Armando Mateus Rojas, Msc.
Ing. Nestor Ivan Ospina, Msc.
Ing. Oscar Mauricio Gélvez Lizarazo, Msc.

GED (Grupo de Estudio y Desarrollo en Robótica)
Facultad de Ingeniería Electrónica
División de Ingenierías
Bogotá, abril de 2026

1. Introducción

El presente informe documenta el avance correspondiente a la **Semana 12** del cronograma de actividades. Esta semana representa el cierre formal de la **Fase 3 (Diseño conceptual de la solución)**, conforme al cronograma definido en el anteproyecto, y consolida la transición hacia la Fase 4 (Desarrollo progresivo de la solución).

El trabajo de esta semana estuvo orientado a resolver definitivamente el bloqueo técnico identificado durante las semanas S10 y S11 respecto a la simulación de la plataforma robótica. Como resultado, se logró por primera vez la ejecución funcional del modelo del vehículo en un entorno de simulación compatible con ROS 2 Humble, lo cual valida experimentalmente la viabilidad de la arquitectura propuesta y desbloquea el inicio de la fase de implementación.

2. Objetivos de la Semana

- Resolver el bloqueo técnico de simulación identificado en las semanas anteriores mediante la adopción de una plataforma de software de código abierto compatible con ROS 2 Humble.
- Validar experimentalmente el funcionamiento del vehículo en simulación: spawn, controladores activos, respuesta a comandos de velocidad y publicación de odometría.
- Cerrar formalmente la Fase 3 mediante la consolidación del diagrama maestro de arquitectura, la tabla de módulos del sistema y la descripción del escenario operativo de relevo.
- Documentar las decisiones técnicas y los procedimientos de migración aplicados sobre el repositorio base, dejando trazabilidad para fases posteriores.

3. Decisión técnica: cambio de plataforma de simulación

3.1. Antecedentes y limitaciones del enfoque DonkeyCar

Como se documentó en el informe de la Semana 10, el ecosistema DonkeyCar planteado inicialmente en el anteproyecto presenta limitaciones técnicas relevantes para el alcance del proyecto. Las dificultades identificadas fueron las siguientes:

- **Desactualización del ecosistema:** El proyecto DonkeyCar no ha recibido actualizaciones activas que aseguren su compatibilidad con versiones recientes de ROS 2 ni con las herramientas de simulación estándar del ecosistema robótico actual.
- **Simulador propio no integrable:** El simulador nativo de DonkeyCar (basado en Unity) no ofrece soporte para integración con ROS 2 Humble, lo cual impide explorar funcionalidades como mapeo, localización (SLAM) y planificación de trayectorias dentro del entorno de simulación esperado por el proyecto.
- **Inexistencia de modelo URDF mantenido:** Tras una búsqueda exhaustiva en repositorios públicos no se encontró un modelo URDF de DonkeyCar funcional y mantenido que pudiera utilizarse en Gazebo con cinemática Ackermann en ROS 2 Humble.

Estas limitaciones, que se evidenciaron progresivamente durante las semanas S9 a S11, representaban un riesgo crítico para el cumplimiento del cronograma, dado que la Fase 4 (Desarrollo progresivo de la solución) requiere un entorno de simulación funcional como punto de partida para la implementación de los nodos de navegación, percepción y coordinación.

3.2. Adopción del modelo AWS DeepRacer con hardware abierto

Como respuesta a las limitaciones anteriores, se decidió adoptar el modelo del vehículo **AWS DeepRacer** como base de simulación, manteniendo el espíritu de bajo costo y código abierto del proyecto original. La decisión se sustenta en los siguientes elementos:

- **Geometría compatible:** El AWS DeepRacer es un vehículo Ackermann a escala 1/18

con dos cámaras RGB y LiDAR, configuración que se ajusta a los requisitos sensoriales y dimensionales planteados en el anteproyecto para la operación en pasillos interiores.

- **Stack open source mantenido:** AWS publicó en código abierto el repositorio `aws-deepracer/aws-d` [1], que incluye el modelo URDF (`deepracer_description`), el plugin de simulación Ackermann (`deepracer_gazebo`) y los archivos de configuración para Gazebo Classic.
- **Reemplazo de la electrónica de control:** Para mantener la filosofía de hardware abierto del proyecto, la unidad de cómputo original del DeepRacer (placa Intel Atom) será reemplazada por una **Raspberry Pi 4**, manteniendo la geometría mecánica del chasis pero permitiendo la ejecución del stack ROS 2 Humble en una plataforma accesible y documentada por la comunidad.
- **Compatibilidad con la planificación cinemática prevista:** El modelo soporta la integración con Nav2, TEB Local Planner y Smac Planner, planificadores que ya habían sido seleccionados en la Semana 11 para la lógica de navegación y retorno.

Esta decisión no modifica el alcance ni los objetivos del proyecto definidos en el anteproyecto. Mantiene los principios fundamentales (cinemática Ackermann, bajo costo, código abierto, dos agentes coordinados) y resuelve el bloqueo técnico identificado en las semanas anteriores.

4. Migración técnica: AWS DeepRacer en ROS 2 Humble

4.1. Contexto de la migración

El stack oficial de AWS DeepRacer fue desarrollado para **ROS 2 Foxy** sobre Ubuntu 20.04 [2]. Dado que el proyecto utiliza **ROS 2 Humble** sobre Ubuntu 22.04 (versión LTS con soporte hasta 2027), fue necesario aplicar una serie de parches sobre el repositorio para asegurar la compatibilidad. La migración se realizó de forma controlada, paquete por paquete, para identificar y aislar cualquier incompatibilidad de la API.

4.2. Parches aplicados

Los parches realizados sobre el repositorio se resumen en la Tabla 1. Estos cambios fueron necesarios debido a la evolución de las APIs de `ros2_control` entre las distribuciones Foxy y Humble [3, 4].

Tabla 1: Parches aplicados al repositorio `aws-deepracer/aws-deepracer` para compatibilidad con ROS 2 Humble.

Archivo	**Cambio anterior (Foxy)**	**Cambio aplicado (Humble)**	**Justificación**
`deepracer_spawn.launch.py`	`'--set-state', 'start'`	`'--set-state', 'active'`	API `ros2_control`
`teleop_test_launch.py`	`'--set-state', 'start'`	`'--set-state', 'active'`	API `ros2_control`
`agent_control.yaml`	`joint_state_controller/JointStateController`	`joint_state_broadcaster/JointStateBroadcaster`	Renombre de paquete en Humble

4.3. Compilación e instalación

Los seis paquetes que componen el stack del DeepRacer (`deepracer_interfaces_pkg`, `deepracer_drive_plu` `deepracer_description`, `cmdvel_to_servo_pkg`, `enable_deepracer_nav_pkg` y `deepracer_bringup`) fueron compilados de forma incremental con `colcon build` sobre ROS 2 Humble, sin errores de compilación. El plugin C++ de cinemática Ackermann (`deepracer_drive_plugin`) presentó únicamente advertencias menores asociadas a cabeceras deprecadas (`tf2_geometry_msgs.h`), que no afectan la funcionalidad.

5. Validación experimental

Una vez completada la migración, se ejecutaron pruebas de validación para confirmar el funcionamiento integral del sistema en simulación.

5.1. Spawn del vehículo en Gazebo

Se lanzó la simulación con el archivo `deepracer_sim.launch.py` sobre un mundo vacío estándar de Gazebo Classic 11. El vehículo fue spawneado correctamente y los sensores LiDAR y de cámara comenzaron a publicar inmediatamente, como se observa en la Figura 1. La simulación se mantuvo estable a 62 FPS con un *Real Time Factor* de 1.00.

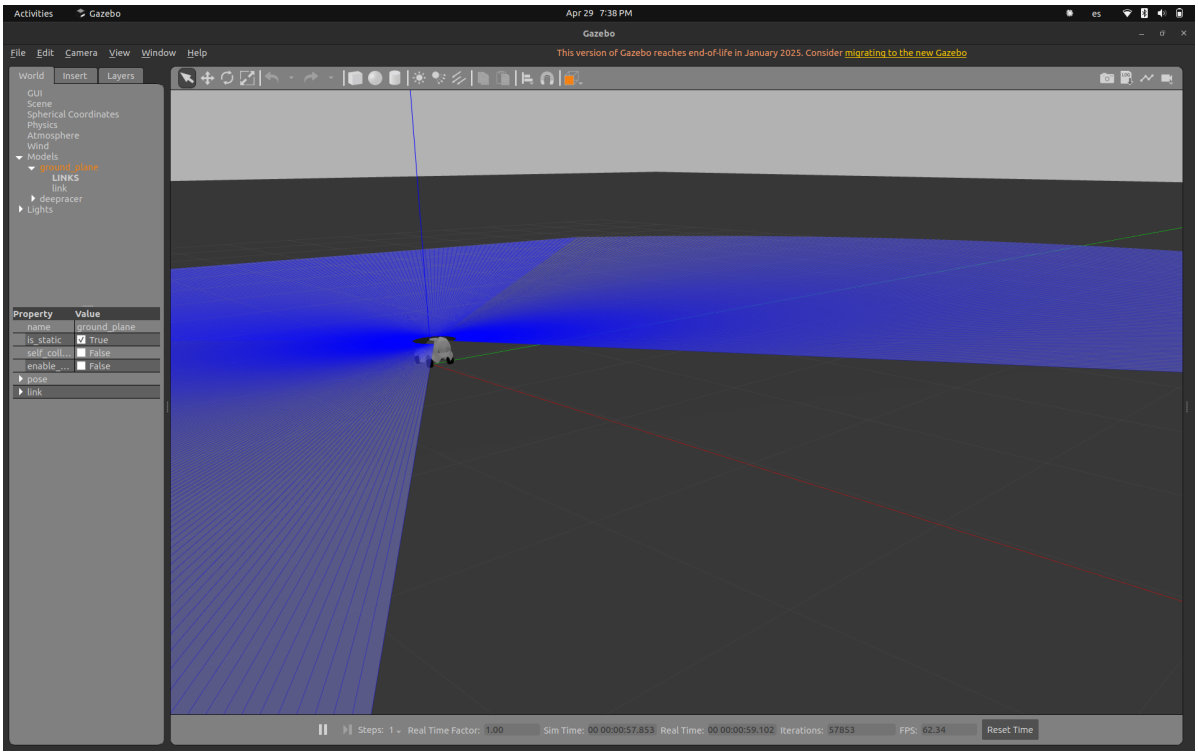

Figura 1: Vehículo AWS DeepRacer spawneado en Gazebo Classic 11. Se observan los rayos del LiDAR activo (cono azul) y la jerarquía de modelos cargados.

5.2. Activación de controladores ROS 2 Control

Tras los parches aplicados, los siete controladores definidos en `agent_control.yaml` fueron cargados exitosamente en estado `active`: `joint_state_broadcaster`, los cuatro controladores de velocidad de las ruedas y los dos controladores de posición de los hinges de dirección. La verificación se realizó mediante `ros2 control list_controllers`, confirmando que todos los componentes del control Ackermann están operativos.

5.3. Respuesta a comandos de velocidad

Se publicaron mensajes `geometry_msgs/Twist` en el tópico `/cmd_vel` para verificar la respuesta del plugin de cinemática Ackermann (Figura 2). El vehículo respondió a los comandos de velocidad lineal y angular publicados de forma secuencial: primero un comando con velocidad lineal positiva, que produjo un avance del vehículo, y posteriormente un comando con componente angular, que generó un cambio en la dirección del movimiento.

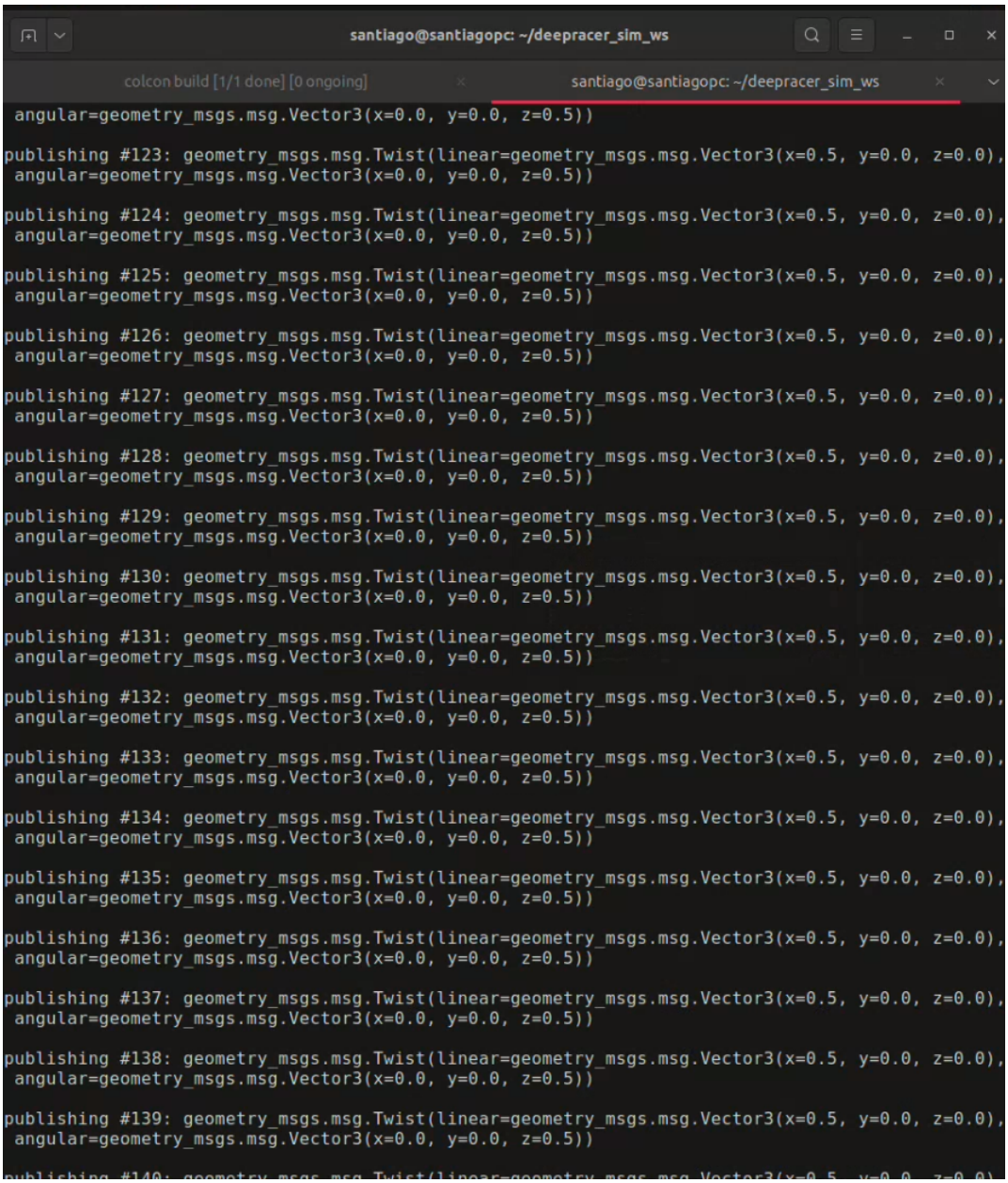

Figura 2: Publicación continua de mensajes `Twist` en el tópico `/cmd_vel` durante una de las pruebas de respuesta a comandos de velocidad.

5.4. Publicación de odometría

El plugin `gazebo_ros_deepracer_drive` publicó datos coherentes en el tópico `/odom`, con una velocidad lineal medida de aproximadamente 0.40 m/s frente a un comando de 0.5 m/s, lo cual es consistente con la respuesta dinámica esperada del controlador PID y la fricción del modelo. La Figura 3 presenta un mensaje completo de odometría capturado durante

el movimiento del vehículo. Las covarianzas reportadas son coherentes con un modelo 2D estándar (valores pequeños en x, y, yaw; valores bloqueados en z, roll, pitch).

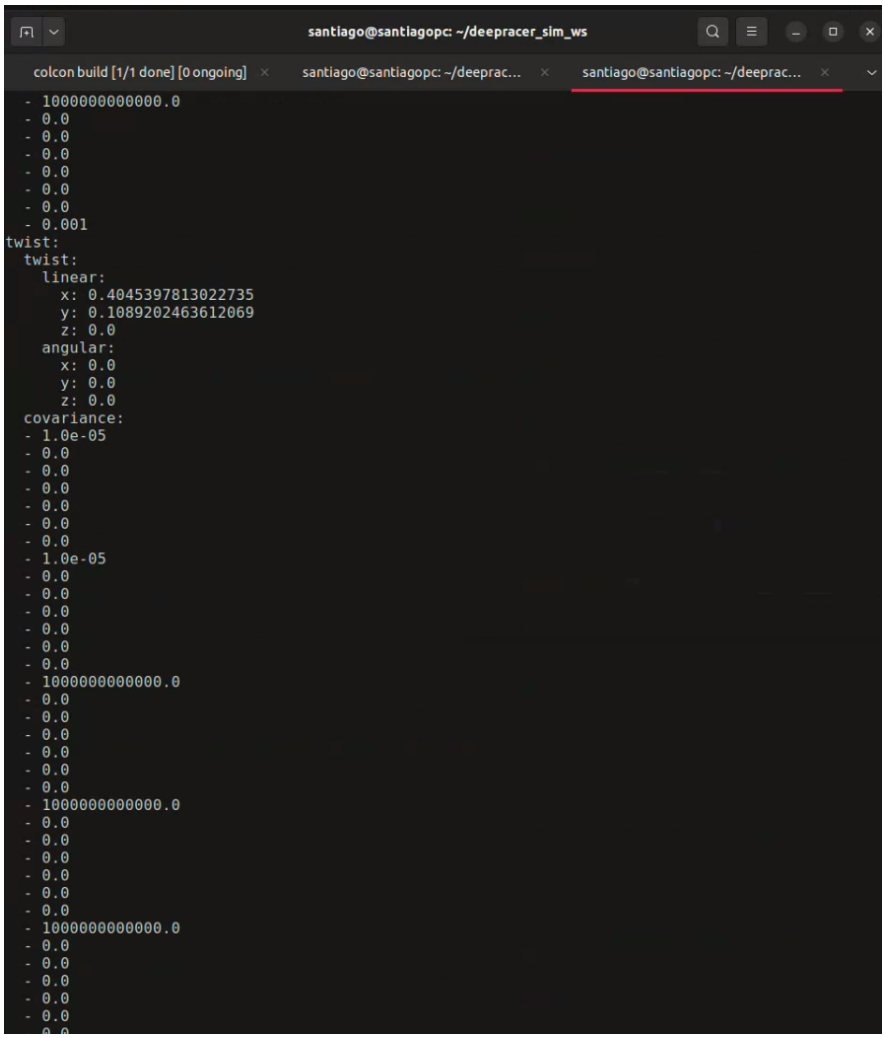

Figura 3: Salida de `ros2 topic echo /odom --once` mostrando la velocidad real medida y las covarianzas asociadas a un modelo de odometría 2D.

6. Cierre conceptual de la Fase 3

6.1. Diagrama maestro de arquitectura

La arquitectura del sistema colaborativo se consolida en el diagrama de la Figura 4. El sistema está compuesto por dos agentes robóticos (uno por piso), un servidor central de coordinación, una interfaz HRI web y la zona de transición vertical (escaleras) que actúa como punto de

relevo entre los agentes.

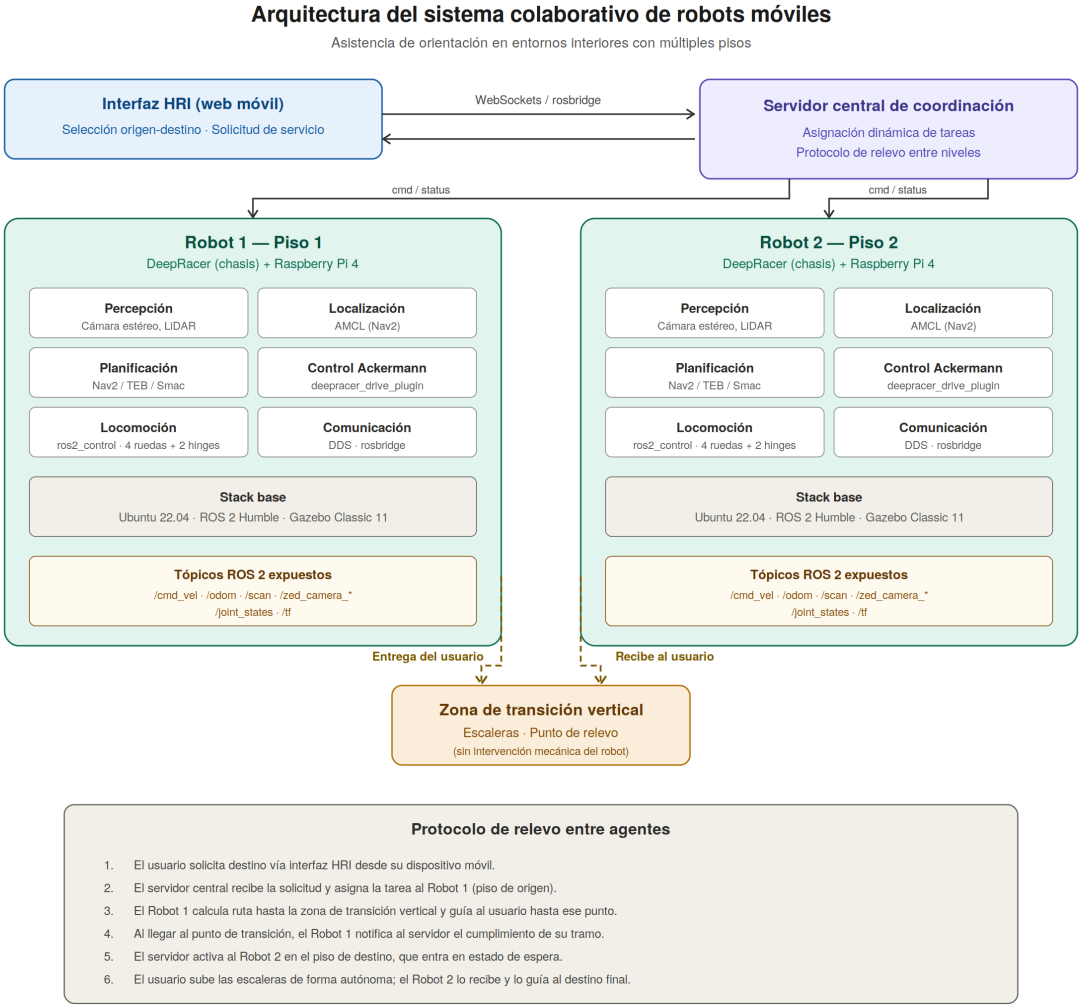

Figura 4: Diagrama maestro de la arquitectura del sistema colaborativo. Los dos agentes DeepRacer operan en pisos distintos coordinados por un servidor central, con la zona de transición vertical como punto de relevo de la guía al usuario.

6.2. Tabla de módulos del sistema

La descomposición funcional prevista del sistema en módulos se presenta en la Tabla 2. Esta tabla refleja la arquitectura conceptual definida para la Fase 3 y orienta el desarrollo de las fases posteriores; las tecnologías marcadas como previstas corresponden a candidatas seleccionadas durante esta etapa de diseño y podrán ajustarse durante la implementación. Se prevé que los dos agentes compartan la misma arquitectura interna y que las diferencias

entre ellos se gestionen mediante *namespaces* ROS 2 (`/robot1`, `/robot2`) y configuración por agente, esquema cuya implementación corresponde a la Fase 4.

Tabla 2: Módulos funcionales del sistema colaborativo (arquitectura prevista).

Módulo	**Función**	**Tecnología prevista**
Percepción	Adquisición sensorial: cámara y LiDAR	ROS 2, `sensor_msgs`
Localización	Estimación de pose en el mapa del piso	Por definir (candidato: AMCL / Nav2)
Planificación	Cálculo de rutas hasta destino o relevo	Por definir (candidatos: Nav2, TEB, Smac)
Control	Conversión de Twist a comandos Ackermann	`deepracer_drive_plugin` (validado)
Locomoción	Actuación física sobre ruedas y dirección	`ros2_control` (validado)
Comunicación local	Mensajería entre nodos del agente	ROS 2 DDS
Comunicación externa	Enlace con servidor y HRI	Por definir (candidato: `rosbridge_suite`)
Coordinación (servidor)	Asignación dinámica de tareas y relevo	Servidor central, protocolo de relevo
Interfaz HRI	Solicitud de destino por parte del usuario	Web móvil (HTML/JS)

6.3. Escenario operativo: protocolo de relevo

El funcionamiento conceptual del sistema durante una solicitud de guía se describe a continuación, asumiendo que el usuario se encuentra en el Piso 1 y desea llegar a un destino en el Piso 2:

1. El usuario accede a la interfaz HRI web desde su dispositivo móvil y selecciona el destino deseado.
2. La interfaz envía la solicitud al servidor central a través de WebSockets / rosbridge.
3. El servidor consulta el estado de los agentes disponibles y asigna la tarea al **Robot 1** (piso de origen). El **Robot 2** entra en estado de espera activa en el piso de destino.
4. El Robot 1 calcula una ruta hasta la zona de transición vertical (escaleras) usando Nav2 y guía al usuario hasta ese punto.
5. Al llegar a la zona de transición, el Robot 1 notifica al servidor el cumplimiento de su tramo. El servidor activa al Robot 2.
6. El usuario sube las escaleras de forma autónoma y es recibido por el Robot 2 en el piso de destino.
7. El Robot 2 guía al usuario hasta el destino final, mientras el Robot 1 retorna a su base de manera autónoma utilizando los planificadores TEB / Smac.

Este esquema resuelve la discontinuidad espacial entre niveles sin requerir hardware especializado para superar barreras arquitectónicas (escaleras o ascensores), manteniendo la simplicidad mecánica del agente individual y trasladando la complejidad al plano de la coordinación inter-robot vía servidor central.

7. Estado del cronograma y trabajo pendiente

7.1. Estado actual de actividades

El estado de las actividades correspondientes a la Fase 3 se resume en la Tabla 3. Con el cierre de esta semana, las cuatro actividades planificadas para la Fase 3 quedan completadas dentro del rango temporal previsto en el cronograma del anteproyecto (S10–S13).

Tabla 3: Estado de actividades — Fase 3.

Actividad	**Estado**	**Observación**
Definición de arquitectura general	Completado	Diagrama maestro consolidado.
Identificación de módulos	Completado	Tabla de módulos formalizada.
Representaciones conceptuales	Completado	Escenario de relevo descrito.
Definición conceptual del funcionamiento	Completado	Protocolo de 7 pasos definido.
Validación experimental de plataforma	Completado	DeepRacer funcional en ROS 2 Humble.

7.2. Trabajo previsto para semanas siguientes

De acuerdo con el cronograma del anteproyecto, las actividades correspondientes a la **Fase 4 (Desarrollo progresivo de la solución, S13–S17)** son las siguientes, y se dejan explícitamente fuera del alcance de este entregable:

- **S13–S14:** Modelado del entorno virtual de la Universidad Santo Tomás en Gazebo, recreando los pasillos y descansos de escaleras a escala 1:1.
- **S13–S15:** Instanciación simultánea de los dos agentes en simulación mediante *namespaces* ROS 2 (`/robot1`, `/robot2`) y validación de la coexistencia de los stacks de control.
- **S13–S16:** Implementación de los nodos del servidor central de coordinación y del protocolo de relevo.

- **S15–S17:** Implementación de la interfaz HRI web y su integración con el servidor mediante `rosbridge_suite`.

8. Conclusiones de la Semana

La Fase 3 del proyecto se cierra con el bloqueo técnico de simulación resuelto y la arquitectura del sistema consolidada conceptualmente. La adopción del stack open source de AWS DeepRacer, complementada con el reemplazo de la unidad de cómputo por una Raspberry Pi 4, permite mantener los principios de bajo costo y código abierto del anteproyecto al mismo tiempo que asegura la viabilidad técnica del desarrollo en ROS 2 Humble.

La validación experimental realizada esta semana confirma que la cadena completa de control y simulación es funcional: desde la publicación de comandos de velocidad hasta la respuesta física del modelo y la lectura de odometría. Esto establece una base técnica sólida para el inicio inmediato de la Fase 4 en la Semana 13, donde se procederá con el modelado del entorno universitario y la instanciación simultánea de los dos agentes que permitirá validar el comportamiento colaborativo del sistema.

Referencias

[1] AWS DeepRacer, "aws-deepracer: Simulation artifacts, core components and configuration files to integrate AWS DeepRacer device with ROS Navigation stack," GitHub repository, 2021. [Online]. Available: `https://github.com/aws-deepracer/aws-deepracer`

[2] AWS Open Source Blog, "AWS DeepRacer is now open source and ready to hit the road with ROS 2," Apr. 2021. [Online]. Available: `https://aws.amazon.com/blogs/opensource/aws-deepracer-is-now-open-source-and-ready-to-hit-the-road-with-ros-2/`

[3] Open Robotics, "ROS 2 Humble Hawksbill Documentation," [Online]. Available: `https://docs.ros.org/en/humble/`

[4] ros2_control Working Group, "ros2_control: A flexible framework for the integration of robot hardware and controllers," [Online]. Available: https://control.ros.org/humble/